МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Балтийский государственный технический университет «ВОЕНМЕХ» им. Д.Ф. Устинова»
(БГТУ «ВОЕНМЕХ» им. Д.Ф. Устинова)

Кафедра Н2 Программная инженерия и интеллектуальные системы
шифр наименование кафедры, по которой выполняется работа

Дисциплина **Языки и средства разработки интернет-приложений**
наименование дисциплины

ПРАКТИЧЕСКАЯ РАБОТА
№5

номер задания (при наличии)

Программный интерфейс в стиле REST
Тема: «Склад небольшого магазина»

при наличии указать тему учебно-практической работы и (или) номер варианта

ОБУЧАЮЩИЙСЯ

группы ЗО721

_____ Бахтина Н.А.
подпись фамилия и инициалы

_____
дата сдачи

ПРОВЕРИЛ

_____ Мажайцев Е.В.
подпись фамилия и инициалы

Оценка / балльная оценка _____

_____
дата проверки

г. Санкт-Петербург
2026 г.

Цель работы: научиться проектировать и реализовывать программный интерфейс в стиле REST, документировать его по спецификации OpenAPI и проверять автоматически тестами.

Задание

Часть 1. Проектирование

1. Уточните перечень ресурсов API из ТЗ. Адреса ресурсов – существительные во множественном числе, вложенность не глубже двух уровней: /orders/15/items, а не /getOrderItems.

2. Методы используются по назначению: GET – чтение, POST – создание, PUT или PATCH – изменение, DELETE – удаление. GET не изменяет данные.

3. Коды ответа соответствуют ситуации, см. таблицу ниже.

Ситуация	Код	Примечание
Данные получены	200	
Ресурс создан	201	Заголовок Location с адресом нового ресурса
Ресурс удален, тело ответа не нужно	204	
Данные запроса не прошли проверку	400 или 422	FastAPI по умолчанию отвечает 422
Запрос без аутентификации	401	
Недостаточно прав	403	
Ресурс не найден	404	
Конфликт, например нарушение уникальности	409	

Часть 2. Реализация

1. Все обработчики API находятся под префиксом версии, например /api/v1.

2. Списки поддерживают постраничный вывод, фильтрацию и сортировку через параметры запроса.

3. Ошибки возвращаются в едином формате. Рекомендуется формат Problem Details по RFC 9457 с типом содержимого application/problem+json.

4. Аутентификация по токену в заголовке Authorization: Bearer.

5. Настроен CORS: разрешен адрес вашего клиента, а не любой адрес.

6. Клиент из ПР 2 работает через эту версию API.

Часть 3. Документация и проверка

1. Спецификация OpenAPI доступна по адресу документации (в FastAPI - /docs). У каждого обработчика есть описание, схемы запроса и ответа, перечень возможных кодов ответа.

2. Не менее десяти автоматических тестов (pytest с TestClient или httpx): успешные сценарии и ответы 401, 403, 404, 422.

3. В репозитории лежит коллекция запросов для ручной проверки: Postman, Bruno или файл .http.

Выполнение работы

1. Проектирование и реализация REST API

В соответствии с заданием, все маршруты API имеют префикс версии /api/v1. Для соблюдения принципов REST были использованы правильные методы (GET для чтения, POST для создания, PUT для изменения, DELETE для удаления). Списки поддерживают пагинацию (skip, limit), фильтрацию (search, month) и сортировку (sort_by, order) через параметры запроса.

На рисунке 1 представлен код с настройкой импортов, CORS и JWT – аутентификации в файле «main.py», а также общая структура проекта слева.

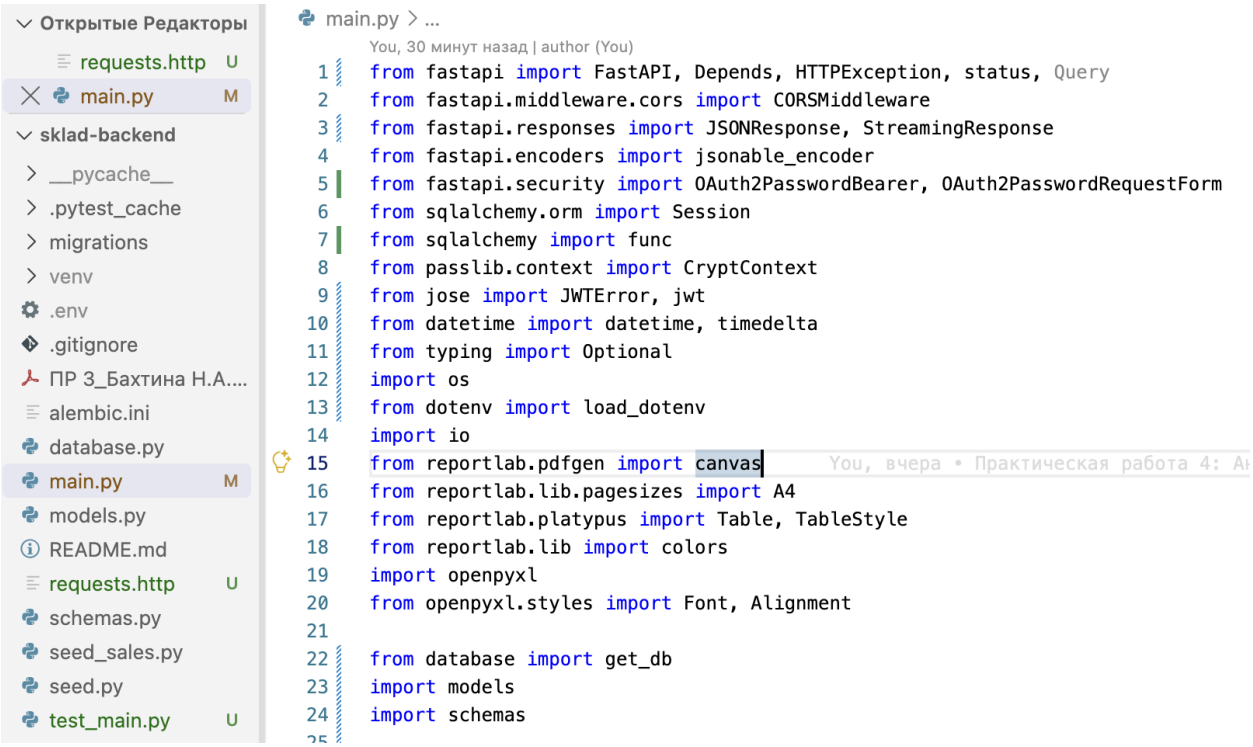


Рисунок 1 – Настройка JWT – аутентификации и CORS в main.py

На рисунке 2 представлен код основного маршрута авторизации.

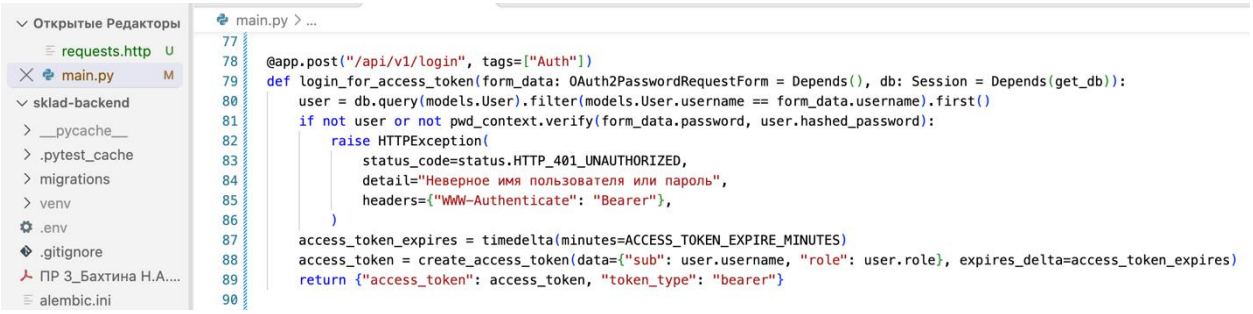


Рисунок 2 – Реализация маршрута авторизации и выдачи токена

2. Коды ответов и безопасность

В коде были заданы правильные HTTP-коды ответов: «201» при создании ресурса, «204» при успешном удалении, «401» при отсутствии или неверном токене, «404» при ненайденном ресурсе, «409» при конфликте (попытке создать дубликат пользователя), «403» при недостатке прав доступа.

На рисунке 3 представлена обновленная документация Swagger UI с группами методов и иконками замков, обозначающими защищенные маршруты. Чтобы работать с API без чтения кода будет достаточно полной «OpenAPI - спецификации».

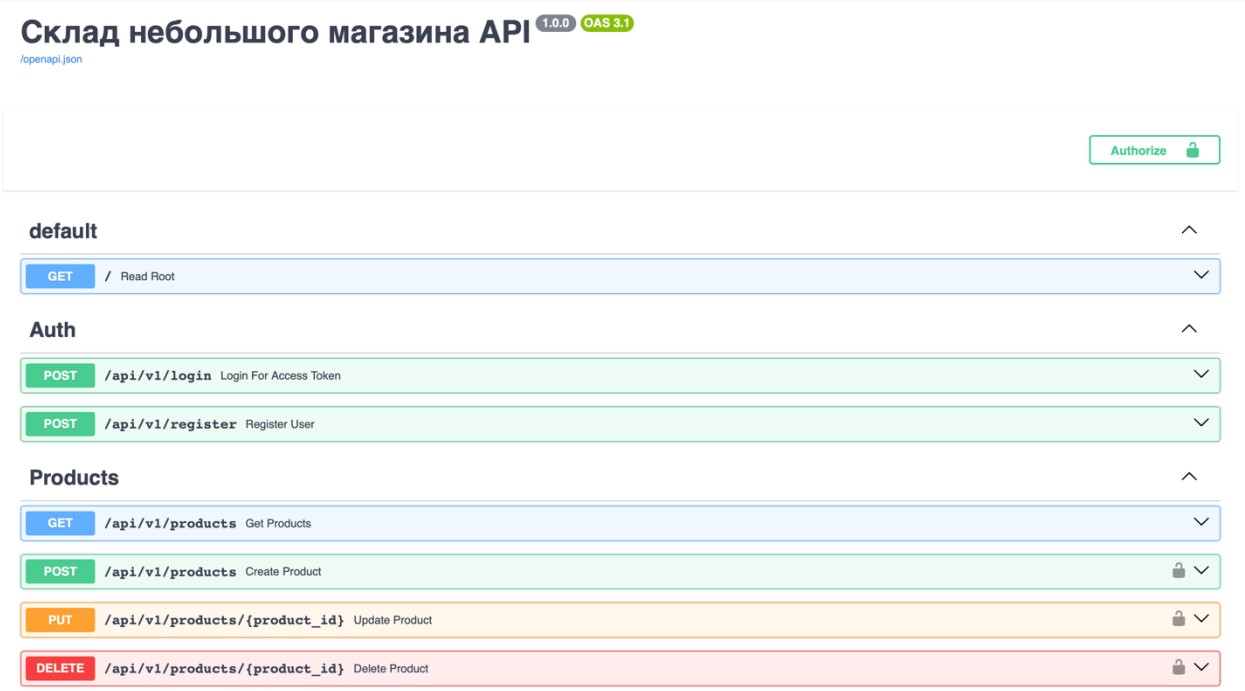


Рисунок 3 – Документация API (Swagger UI) с разделением на теги и защищенными маршрутами (группы «default», «Auth» и «Products»)

Сверху, в правом углу страницы, видно кнопку «Authorize», которая позволяет авторизоваться прямо в Swagger, вставив токен, после чего замочки на методах откроются.

На рисунке 4 представлены группы разделов «Analytics» и «Documents» документации API.

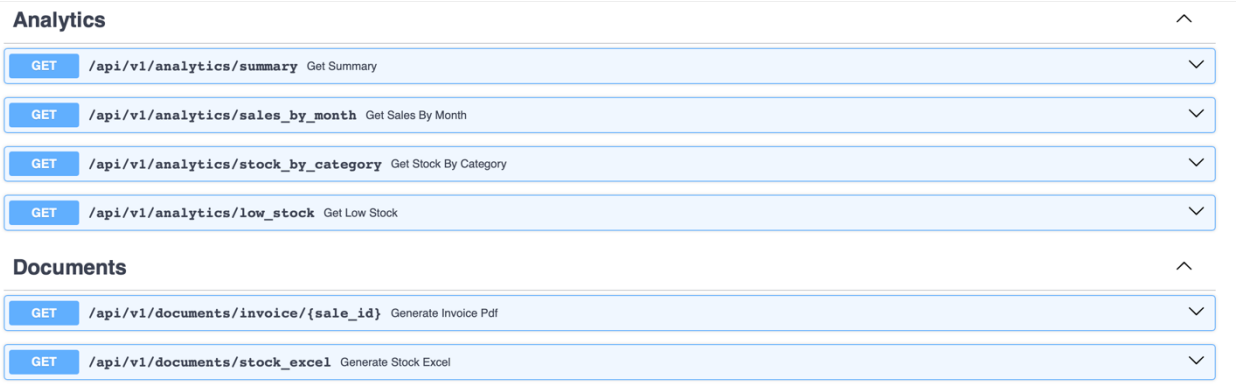


Рисунок 4 – Документация API (Swagger UI), группы «Analytics» и «Documents»

Все методы аккуратно разбиты на «Auth», «Products», «Analytics» и «Documents». Это делает документацию удобной. Также видно, что методы создания, изменения и удаления товаров защищены токеном (без него вернется «401»). Указана версия API «1.0.0»

На рисунке 5 представлены схемы данных документации API.



Рисунок 5 – Схемы данных (Schemas) в документации OpenAPI

3. Автоматическое тестирование

Для проверки работоспособности API были написаны автоматические тесты с использованием библиотеки «pytest» и «TestClient». На рисунке 6 представлена успешная установка библиотеки «python-multipart». Тесты проверяют как успешные сценарии (авторизация, получение товаров, создание товаров), так и ошибочные (попытка создать товар без токена «401», обновление несуществующего товара «404», невалидные данные «422», а также проверку прав доступа «403»). Одиннадцатый тест проверяет разграничение ролей: пользователь с ролью «Кладовщик» не может удалить

товар, так как эта операция доступна только Администратору, поэтому сервер возвращает ошибку «403».

```
(venv) MacBook-Pro-macally:sklad-backend macally$ pip install python-multipart
Collecting python-multipart
  Downloading python_multipart-0.0.33-py3-none-any.whl.metadata (6.3 kB)
Downloading python_multipart-0.0.33-py3-none-any.whl (32 kB)
Installing collected packages: python-multipart
Successfully installed python-multipart-0.0.33

[notice] A new release of pip is available: 24.0 -> 26.3
[notice] To update, run: pip install --upgrade pip
(venv) MacBook-Pro-macally:sklad-backend macally$
```

Рисунок 6 – Установка библиотеки для тестов «python-multipart»

Все 11 тестов успешно выполняются. На рисунке 7 представлен код тестов, а на рисунке 8 – результат их выполнения.

test_main.py > ...

```python
from fastapi.testclient import TestClient
from main import app

client = TestClient(app)

# Переменная для хранения токена
auth_token = ""

def test_read_root():
    response = client.get("/")
    assert response.status_code == 200

def test_login_success():
    global auth_token
    response = client.post("/api/v1/login", data={"username": "admin", "password": "secret123"})
    assert response.status_code == 200
    assert "access_token" in response.json()
    auth_token = response.json()["access_token"]

def test_login_fail():
    response = client.post("/api/v1/login", data={"username": "admin", "password": "wrongpassword"})
    assert response.status_code == 401
    assert response.json()["detail"] == "Неверное имя пользователя или пароль"

def test_get_products():
    response = client.get("/api/v1/products")
    assert response.status_code == 200
    assert isinstance(response.json(), list)

def test_create_product_without_token():
    response = client.post("/api/v1/products", json={"name": "Тестовый товар", "category_id": 1, "price": 100.0})
    assert response.status_code == 401

def test_create_product_with_token():
    headers = {"Authorization": f"Bearer {auth_token}"}
    response = client.post(
        "/api/v1/products",
        json={"name": "Тестовый товар", "category_id": 1, "price": 100.0, "current_stock": 10, "min_stock": 5},
        headers=headers
    )
    assert response.status_code == 201
    assert response.json()["name"] == "Тестовый товар"

```

```python
def test_update_nonexistent_product():
    response = client.put(
        "/api/v1/products/99999",
        json={"name": "Обновленный товар", "category_id": 1, "price": 100.0, "current_stock": 10, "min_stock": 5},
        headers={"Authorization": f"Bearer {auth_token}"}
    )
    assert response.status_code == 404
    assert response.json()["detail"] == "Товар не найден"

def test_delete_product_without_token():
    response = client.delete("/api/v1/products/1")
    assert response.status_code == 401

def test_delete_product_with_token():
    # Получаем список, чтобы узнать ID последнего созданного товара
    response_list = client.get("/api/v1/products?limit=100")
    product_id = response_list.json()[-1]["id"]

    response = client.delete(
        f"/api/v1/products/{product_id}",
        headers={"Authorization": f"Bearer {auth_token}"}
    )
    assert response.status_code == 204

def test_register_invalid_data():
    response = client.post("/api/v1/register", json={})
    assert response.status_code == 422


def test_delete_product_forbidden():
    # Регистрируем кладовщика
    client.post("/api/v1/register", json={"username": "worker_test", "password": "123", "role": "worker"})

    # Логинимся под ним
    response_login = client.post("/api/v1/login", data={"username": "worker_test", "password": "123"})
    worker_token = response_login.json()["access_token"]

    # Пытаемся удалить товар (должны получить 403)
    response = client.delete(
        "/api/v1/products/1",
        headers={"Authorization": f"Bearer {worker_token}"}
    )
    assert response.status_code == 403
    assert response.json()["detail"] == "Недостаточно прав для удаления"
```

Рисунок 7 – Автоматические тесты API (test_main.py)

```
(venv) MacBook-Pro-macally:sklad-backend macally$ pytest
============================= test session starts ==============================
platform darwin -- Python 3.12.3, pytest-9.2.0, pluggy-1.7.0
rootdir: /Users/macally/Desktop/sklad-backend
plugins: anyio-4.16.0
collected 11 items

test_main.py ...........                                                 [100%]

=============================== warnings summary ===============================
venv/lib/python3.12/site-packages/fastapi/testclient.py:1
  /Users/macally/Desktop/sklad-backend/venv/lib/python3.12/site-packages/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testc
lient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

venv/lib/python3.12/site-packages/passlib/utils/__init__.py:854
  /Users/macally/Desktop/sklad-backend/venv/lib/python3.12/site-packages/passlib/utils/__init__.py:854: DeprecationWarning: 'crypt' is deprecated and slated for r
emoval in Python 3.13
    from crypt import crypt as _crypt

test_main.py::test_login_success
test_main.py::test_delete_product_forbidden
  /Users/macally/Desktop/sklad-backend/main.py:53: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use
 timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    expire = datetime.utcnow() + expires_delta

test_main.py::test_create_product_with_token
  /Users/macally/Desktop/sklad-backend/main.py:109: PydanticDeprecatedSince20: The `dict` method is deprecated; use `model_dump` instead. Deprecated in Pydantic V
2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at https://errors.pydantic.dev/2.14/migration/
    new_product = models.Product(**product.dict())

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
======================== 11 passed, 5 warnings in 2.61s ========================
(venv) MacBook-Pro-macally:sklad-backend macally$
```

Рисунок 8 – Результат выполнения автоматических тестов API (pytest)

Успешное прохождение всех 11 тестов означает, что сервер работает идеально, он защищен токенами, правильно выдает ошибки и выполняет все CRUD-операции.

Строчки с текстом «DeprecationWarning» - это не ошибки, а предупреждения от Python, что в будущих версиях какие-то функции обновятся.

4. Коллекция запросов для ручной проверки

Для ручной проверки API в репозитории создан файл «requests.http», содержащий готовые запросы для проверки авторизации, CRUD – операций и получения аналитики. На рисунке 9 представлен фрагмент этого файла.

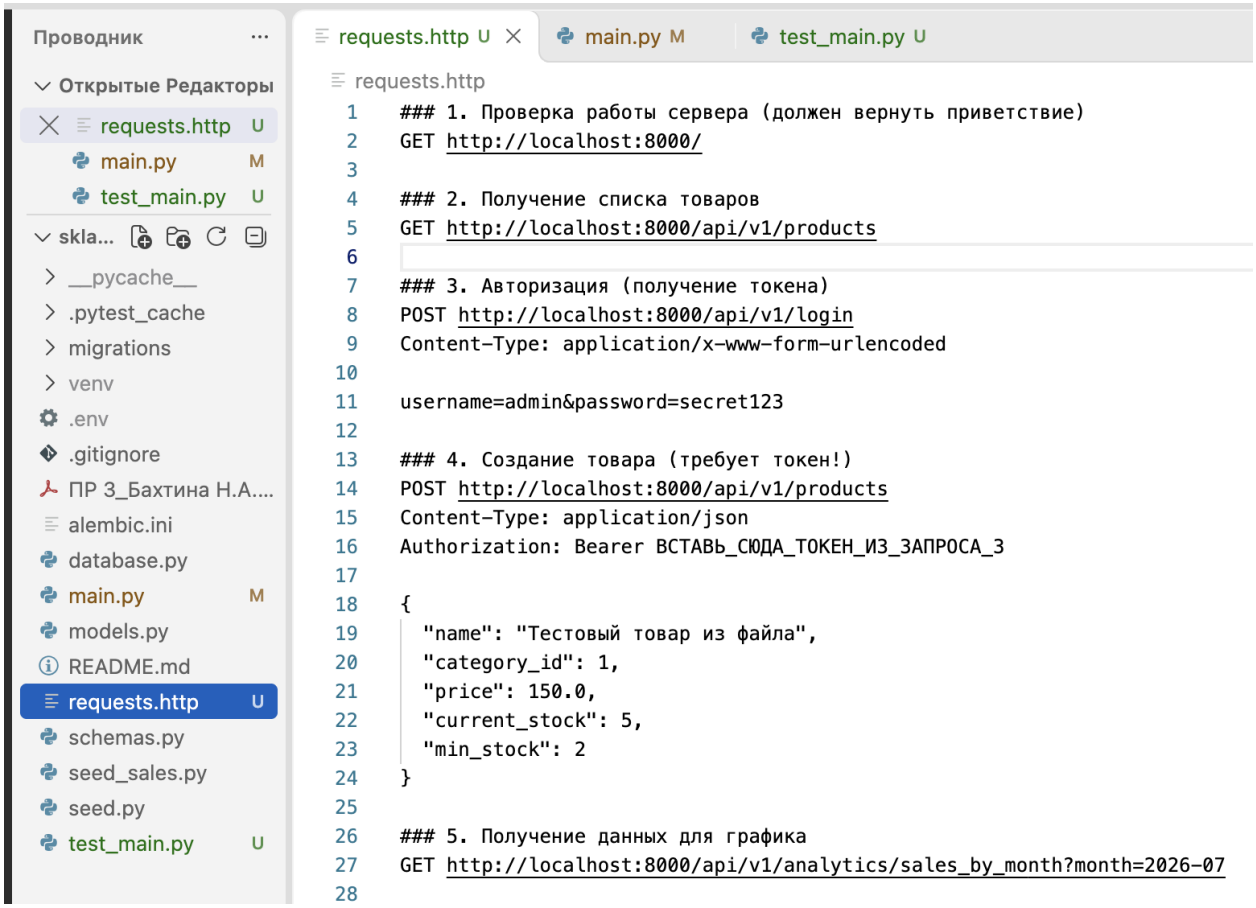


Рисунок 9 – Коллекция запросов для ручной проверки (requests.http)

Отправляем код в репозиторий на GitHub. На рисунке 10 представлена успешная отправка кода.

```
(venv) MacBook-Pro-macally:sklad-backend macally$ git push
Enumerating objects: 11, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 8 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (7/7), 3.65 KiB | 3.65 MiB/s, done.
Total 7 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 2 local objects.
To https://github.com/mcnatusik-design/sklad-backend.git
   681e31e..83f6fb6  main -> main
(venv) MacBook-Pro-macally:sklad-backend macally$
```

Рисунок 10 – Успешная отправка кода (команда «git push»

На рисунке 11 представлен обновленный репозиторий, и папка «sklad-backend» с добавленными файлами «test_main.py» и «requests.http».

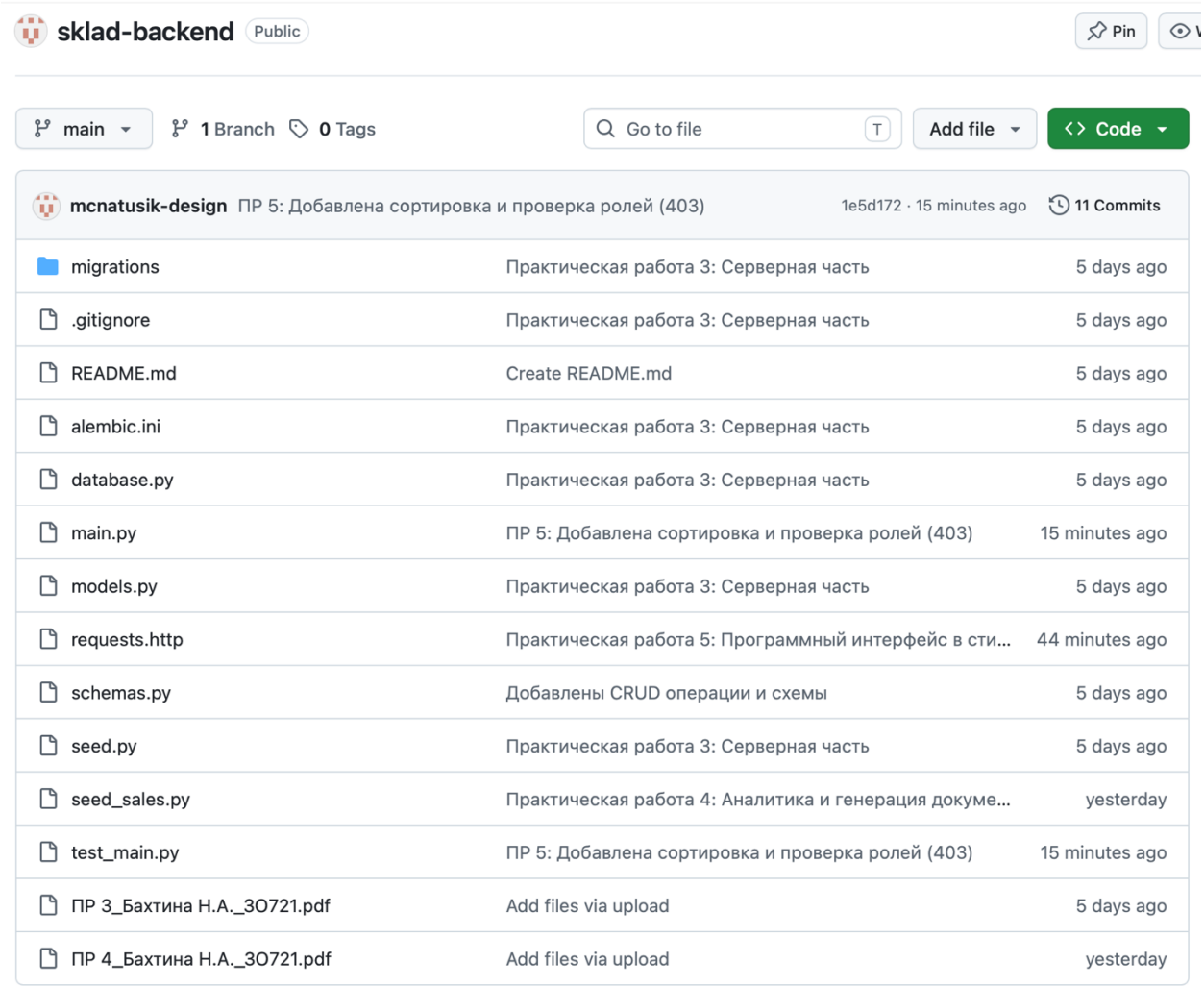


Рисунок 11 – Файловая структура проекта на GitHub

Выводы по практической работе.

В ходе выполнения практической работы были получены навыки проектирования REST API, документирования его с помощью спецификации

OpenAPI (Swagger), настройки JWT – аутентификации и разграничения прав доступа. Также было освоено написание автоматических тестов с использованием «pytest», проверяющих как успешные сценарии работы сервера, так и обработку ошибок.

Исходный код клиентской части, расположенный в репозитории «sklad-magazina» на GitHub доступен по ссылке: https://github.com/mcnatusik-design/sklad-magazina.

Исходный код серверной части, расположенный в репозитории «sklad-backend» на GitHub доступен по ссылке: https://github.com/mcnatusik-design/sklad-backend.